Faculteit Industriële Ingenieurswetenschappen

Campus Gent

Geavanceerde computerarchitectuur: Snake Battle Royale

Auteurs:
Stijn Verhaeghe
Andres Bosman

Opleiding:
Master in de industriële wetenschappen: Elektronica

May 18, 2026

Academiejaar 2025 - 2026

Contents

1 Introductie **2**

2 CPU-Implementatie en Prestatiemetingen **2**
2.1 Gegevensstructuur van de Slang 2
2.2 Logische Functionaliteiten 3
2.3 Visualisatie van snakes via SFML 4
2.4 Werking flow van programma 5
2.5 Prestaties van de CPU 6

3 GPU-Implementatie **7**
3.1 Architecturale Verschuiving 7
3.2 Geheugenbeheer en Optimalisatie 7
3.3 Architecturale verschillen en datastructuren 7
3.4 Basis GPU-implementatie 8
3.4.1 Prestatie 9
3.5 optimalisatie 10
3.5.1 AliveList 10
3.5.2 Fruit constant memory 11
3.5.3 SoA 13
3.5.4 shared memory 15

4 Code **16**
4.1 CPU code 16
4.2 GPU code 22

1 Introductie

In dit project wordt de implementatie van een grootschalige *Snake Battle Royale*-simulatie onderzocht. De bedoeling is om een grote hoeveelheid slangen simultaan in te laden op een speelveld. Binnen deze simulatie moeten de slangen actief op zoek gaan naar fruit om in lengte te groeien, terwijl ze proberen te overleven door andere slangen te elimineren. Een essentieel onderdeel van de simulatie is de bewegingslogica en de botsingsdetectie. De locaties van alle slangen moeten constant worden bijgehouden om te controleren wanneer een slang de rand van het veld raakt of botst met het lichaam van een andere slang, wat resulteert in het beëindigen van de simulatie voor die specifieke slang. Om de prestatievoordelen van moderne computerarchitecturen aan te tonen, wordt de game eerst volledig op de CPU gebouwd. Deze versie dient als referentiepunt om de rekentijd te meten. Vervolgens wordt de volledige logica geoptimaliseerd en geïmplementeerd aan de hand van CUDA-kernels. Hierdoor kan de enorme hoeveelheid berekeningen die nodig is voor duizenden slangen gelijktijdig op de GPU worden uitgevoerd, wat een significante schaalvergroting van het spel mogelijk maakt.

2 CPU-Implementatie en Prestatiemetingen

De initiële versie van de *Snake Battle Royale* is ontwikkeld voor de CPU om een baseline te vestigen. In deze architectuur worden alle berekeningen sequentieel uitgevoerd. Dit betekent dat voor elke frame de processor een centrale lus doorloopt waarin de logica voor elke slang afzonderlijk wordt afgehandeld. De snakes moeten volgende taken kunnen uitvoeren:

- Botsingsdetectie
- Bewegings- en Geheugenbeheer
- Consumptie- en Groeilogica

2.1 Gegevensstructuur van de Slang

Om de toestand van elke individuele slang efficiënt te beheren, wordt gebruikgemaakt van een *struct* genaamd `Snake`. Deze structuur fungeert als een centrale container waarin alle relevante eigenschappen van een slang zijn gebundeld, zodat de CPU deze gegevens tijdens de seriële verwerkingslus per entiteit kan raadplegen en bijwerken.

De onderstaande leden maken deel uit van de `Snake`-structuur:

- `body` **(**$std :: vector < sf :: Vector2i >$**):** Dit is een dynamische lijst die alle coördinaten van de lichaamssegmenten bevat. De CPU gebruikt deze lijst om de fysieke vorm van de slang te onthouden en om botsingen met andere segmenten te detecteren. Elk element is een $sf :: Vector2i$, een 2D-vector met gehele coördinaten (x, y).

- `dir` ($sf :: Vector2i$)**:** Deze variabele slaat de huidige bewegingsrichting op. Het dient als invoer voor de navigatielogica om de coördinaten van de volgende stap te berekenen. Hier gebruiken we terug de sf implementatie van een vector aangezien we alleen de x en de y richting nodig hebben.

- `color` ($sf :: Color$)**:** Slaat de unieke kleurgegevens op. Dit is essentieel voor de visualisatiefase, waarbij elke slang een eigen kleur geeft om ze visueel van elkaar te kunnen onderscheiden.

- `alive` (*bool*)**:** Een booleaanse vlag die de status van de slang bijhoudt. Wanneer een botsing wordt gedetecteerd, wordt deze op `false` gezet, waardoor de CPU de slang in volgende berekeningen negeert.

- *score* (*int*)**:** Een teller die het aantal geconsumeerde fruit bijhoudt. Deze waarde wordt direct uitgelezen voor de weergave op het *leaderboard*.

- `id` (*int*)**:** Een uniek identificatienummer voor elke slang. Dit rugnummer maakt het mogelijk om individuele prestaties te volgen en specifieke slangen te identificeren in een populatie van duizenden entiteiten.

2.2 Logische Functionaliteiten

Om het programma te laten werken zijn er meerdere functie aangemaakt:

1. De eerste functie is de **initSnake**: deze vereist drie parameters, de id om te weten over welke snake het gaat, de breedte en de lengte van de snake. Via deze functie worden alle snakes geïnitialiseerd.

2. De volgende functie is de **isSafe**: Via deze methode wordt gecheckt of de snake tegen een muur botst of tegen een andere slang. Deze functie returnt een true of een false.

3. De derde functie is de **decideDirection**: deze gaat de richting van de snake bepalen, de logica is hier heel eenvoudig geïmplementeerd. Er wordt gekeken naar het dichtstbijzijnde fruit in de buurt en daar gaat de snake automatisch naartoe. Dit gebeurd via de volgende code:

Listing 1: logica voor beweging

```
1     if (!snakes[id].alive) return;
2
3     sf::Vector2i head = snakes[id].body[0];
4     sf::Vector2i bestDir = snakes[id].dir;
5     float minDist = 1000000;
6
7     // Check alle 4 richtingen
8     for(auto& d : directions) {
9         // Voorkom dat de slang direct 180 graden
              omdraait
```

```
10          if (d.x == -snakes[id].dir.x && d.y == -snakes[
                id].dir.y) continue;
11
12          sf::Vector2i next = head + d;
13          if (isSafe(next, width, height)) {
14              // Zoek dichtstbijzijnde fruit
15              for(int f = 0; f < NUM_FRUITS; f++) {
16                  float dist = abs(next.x - fruits[f].x) +
                        abs(next.y - fruits[f].y);
17                  if (dist < minDist) {
18                      minDist = dist;
19                      bestDir = d;
20                  }
21              }
22          }
23      }
24      snakes[id].dir = bestDir;
25  }
```

4. De laatste functie is **spawnFruit**: deze functie zorgt ervoor dat als er fruit wordt opgegeten, nieuw fruit inspawnt op een bepaalde positie.

2.3 Visualisatie van snakes via SFML

De visualisatie zoals weergegeven in fig. 1 van de simulatie is gerealiseerd met de SFML-bibliotheek, waarbij de grafische weergave strikt gescheiden is van de onderliggende logica. De spelwereld wordt vertaald van een abstract coördinatensysteem naar een visueel grid, waarbij elke cel ($CELL_SIZE$) vijf pixels beslaat.

De rendering pipeline volgt een gelaagde structuur: eerst wordt de speelruimte gewist en gevuld met de entiteiten (rood fruit en gekleurde slangen), waarna een semi-transparante UI-overlay wordt getekend. Voor de slangen wordt gebruikgemaakt van sf::RectangleShape, waarbij de kleur dynamisch wordt opgehaald uit de data-structuur van de betreffende entiteit.

Om de gebruikerservaring te verbeteren, bevat de visualisatie een real-time Leaderboard. Hiervoor worden de 10.000 entiteiten elke frame gesorteerd op score, waarna de top 20 via sf::Text in een sidebar wordt weergegeven. De koppeling tussen de logica en visualisatie zorgt ervoor dat de gebruiker direct inzicht krijgt in de prestaties van het algoritme en de status van de populatie.

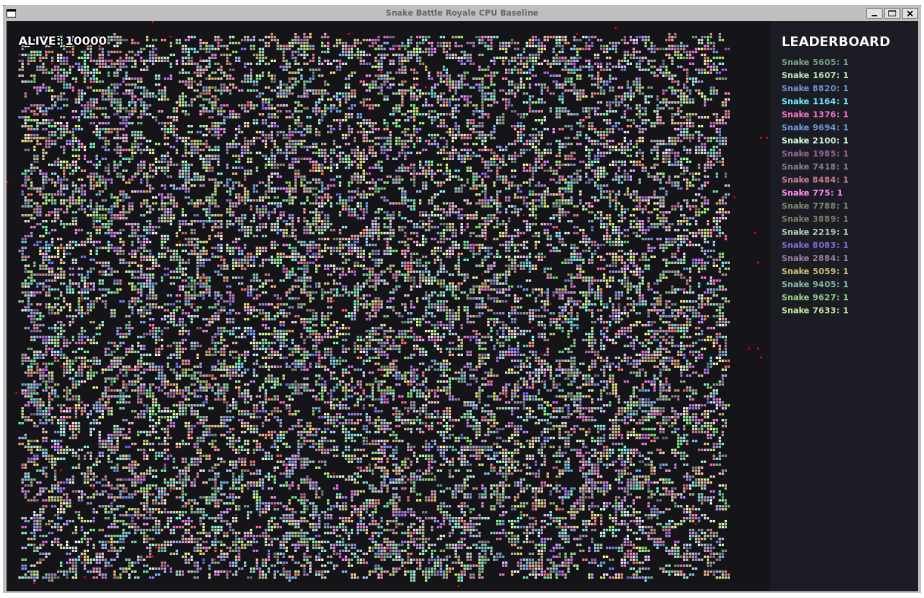


Figure 1: Voorbeeld weergave van de snake game

2.4 Werking flow van programma

De simulatie start met een eenmalige initialisatiefase. Hierbij wordt de speelomgeving klaargezet door 100 stukken fruit op willekeurige locaties te plaatsen en de array van 10.000 slangen te vullen. Elke slang krijgt een startpositie binnen veilige marges, een willekeurige kleur en een beginlengte van drie segmenten om direct herkenbaar te zijn op het grid.

Zodra de initialisatie voltooid is, treedt het programma in de hoofdlus. Deze lus begint met het verwerken van de logica op de CPU. Per actieve slang berekent een algoritme de kortste route naar het dichtstbijzijnde fruit. Voordat een slang daadwerkelijk beweegt, voert de processor een veiligheidscheck uit. Er wordt gecontroleerd of de beoogde stap binnen de muren valt en of er geen botsing plaatsvindt met de lichamen van de duizenden andere slangen. Wanneer een beweging onveilig is, wordt de status van de slang op 'niet levend' gezet, waardoor deze in volgende rondes wordt overgeslagen.

Na de logische verwerking volgt de visualisatiefase via SFML. Het programma wist het vorige frame en tekent alle actieve entiteiten opnieuw op basis van hun actuele grid-coördinaten. Hierbij wordt een UI-overlay toegevoegd met een HUD en een leaderboard dat de top 20 scores weergeeft. Ten slotte wordt het volledige beeld naar de monitor gestuurd en begint de cyclus opnieuw, totdat er slechts één winnaar overblijft en de simulatie herstart.

2.5 Prestaties van de CPU

In het begin van de simulatie leven er 10.000 slangen en dat vraagt veel van de processor. Het duurt dan ongeveer 10 seconden voordat alle slangen één positie zijn opgeschoven. Dit komt doordat de CPU alles serieel verwerkt. Bij elke slang moet de computer controleren of er een botsing is met de muren of met de lichamen van alle andere slangen.

In fig. 2 zie je dit duidelijk terug in de grafiek. De rekentijd (Logic Time) begint erg hoog maar daalt snel zodra er minder slangen overblijven. Dit laat zien dat de belasting niet lineair is. Hoe minder slangen er nog in leven zijn, hoe minder controles de CPU hoeft uit te voeren en hoe sneller de updates elkaar opvolgen.

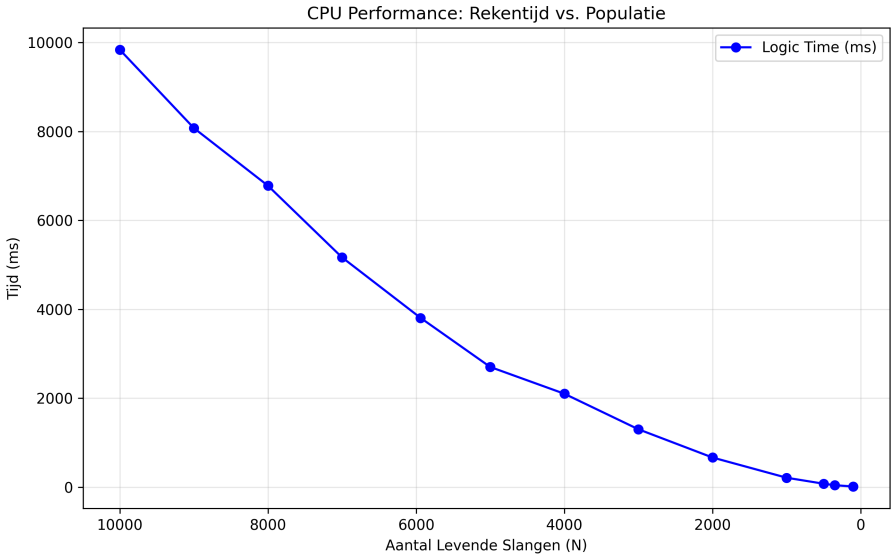


Figure 2: Rekentijd (Logic Time) in ms bij een schaal van 10.000 entiteiten.

3 GPU-Implementatie

De CPU-simulatie bevestigde dat seriële verwerking niet schaalbaar is voor tienduizenden slangen. De bottleneck ligt niet bij de complexiteit van de rekentaken zelf, maar bij de sequentiële uitvoering. Om 10.000 slangen real-time te simuleren, stappen we over naar de parallelle architectuur van de GPU.
We beginnen met een simpele versie en gaan cumulatief meer en meer optimalisatie technieken toepassen. **We vergelijken dus geen technieken met elkaar maar ten opzichte van het niet geimplementeerd te hebben**.

3.1 Architecturale Verschuiving

De overstap naar de GPU vereist een verandering in de manier waarop instructies worden verwerkt

- **CPU (Serieel):** Maakt gebruik van een beperkt aantal kernen die taken in een wachtrij afhandelen. Elke slang moet wachten op de berekening van zijn voorganger, wat leidt tot een cumulatieve rekentijd die kwadratisch groeit met de populatie.
- **GPU (Parallel):** Maakt gebruik van duizenden kleine, gespecialiseerde kernen. Via het **CUDA-platform** wordt elke individuele slang toegewezen aan een eigen **thread**. Hierdoor vinden beweging, logica en botsingsdetectie voor nagenoeg de gehele populatie simultaan plaats.

3.2 Geheugenbeheer en Optimalisatie

De overstap naar de GPU vereist meer dan enkel parallellisatie, de volledige potentie van de hardware kan enkel benut worden door het geheugenbeheer te herzien. In deze implementatie staan drie optimalisaties centraal:

1. **Data-layout:** De transitie van *Array of Structures* (AoS) naar een *Structure of Arrays* (SoA) model. Dit is essentieel voor *coalescing*, waarbij meerdere threads tegelijkertijd aangrenzende geheugenlocaties kunnen aanspreken.
2. **Constant Memory:** Het gebruik van gespecialiseerd geheugen voor data die tijdens de simulatie niet verandert, zoals de fruitposities.
3. **Shared Memory:** Het inzetten van on-chip geheugen voor ultrasnelle communicatie tussen threads binnen hetzelfde block, wat de afhankelijkheid van het tragere globale geheugen vermindert.

3.3 Architecturale verschillen en datastructuren

De CPU maakt gebruik van de SFML bibliotheek en van vectors in plaats van arrays. Dit is niet mogelijk op de GPU. De GPU wilt data dat direct aanspreekbaar is zonder abstractielagen.

Om deze reden is de volledige logica herschreven naar klassieke C-arrays en primitives. De datastructuur van de slang is aangepast om compatibel te zijn met het videogeheugen, waarbij complexe objecten zijn vervangen door eenvoudige arrays:

Listing 2: Aangepaste Snake-structuur voor GPU-compatibiliteit

```
struct Snake {
    int  bodyX[MAX_LEN];
    int  bodyY[MAX_LEN];
    //We werken met een lijst voor de X positie en een lijst voor de Y positie
    //Op deze manier kunnen we de structuur van de vorige code een beetje behouden
    int  length;
    //je kan een maximum lengte instellen waarna de slang wel nog fruit kan eten maar niet meer groter wordt
    int  dirX, dirY;
    //zelfde verhaal voor de richting als voor de positie
    int  r, g, b;
    //kleuren worden nu ook via rgb waarden gedaan in plaats van met de SFML
    bool alive;
    int  score;
    int  id;
};
```

Door de positie en richting op te splitsen in losse integer-arrays, blijft de logica van de oorspronkelijke code behouden, terwijl de data nu wel rechtstreeks naar de GPU gekopieerd kan worden via *cudaMemcpy*.

3.4 Basis GPU-implementatie

De overgang van de CPU naar de GPU-architectuur vereist een transformatie van de seriële logica naar een parallel model. In deze eerste fase wordt de bestaande datastructuur behouden om de werking van de kernels te valideren en een betrouwbare baseline vast te leggen. In plaats van een sequentiële verwerking van de populatie, wordt elke individuele slang toegewezen aan een eigen **thread**. Op deze manier worden taken zoals de controle en de beweging voor de gehele populatie nagenoeg simultaan uitgevoerd door de vele kernen van de grafische kaart.

De simulatie is opgedeeld in gespecialiseerde kernels die de verschillende logische stappen afhandelen. In de eerste fase berekent elke thread parallel de richting voor zijn specifieke slang op basis van de kortste afstand tot het fruit. Vervolgens handelen aparte kernels de fysieke verplaatsing en de interacties met de omgeving af. Omdat duizenden threads tegelijkertijd data kunnen aanpassen, bijvoorbeeld bij het bijhouden van de actuele populatie, wordt er gebruikgemaakt van atomaire operaties. Dit garandeert dat de data-integriteit behouden

blijft en dat er geen race conditions ontstaan wanneer meerdere threads op hetzelfde moment dezelfde geheugenlocatie willen beschrijven.

Een essentieel aspect van deze implementatie is de data-persistentie op het Device. De volledige dataset van de slangen wordt bij de start éénmalig van de Host naar de GPU gekopieerd. Tijdens de actieve simulatie vindt er geen constante datacommunicatie plaats voor de logische berekeningen; enkel voor de visuele weergave wordt de actuele status per frame gedownload. Hoewel deze parallelle aanpak de enorme tijdsvertraging van de CPU-versie elimineert, dient deze opzet in dit stadium vooral als fundament voor verdere verfijningen in het geheugenbeheer en de reken-efficiëntie.

3.4.1 Prestatie

De resultaten van de basis GPU-implementatie, zoals weergegeven in fig. 3, tonen de prestatiewinst van de parallelle uitvoering aan ten opzichte van de eerdere seriële verwerking. Bij een populatie van ongeveer 7.000 actieve slangen bedraagt de rekentijd per frame circa 37 ms. Dit resultaat vormt een directe verbetering ten opzichte van de CPU-metingen, waarbij de rekentijd bij vergelijkbare populaties aanzienlijk hoger lag.

De curve in de grafiek vertoont een nagenoeg lineair verloop naarmate het aantal slangen afneemt. Bij een populatie van ongeveer 3.000 slangen bedraagt de rekentijd nog 15 ms. Wanneer de populatie verder daalt onder de 1.000 entiteiten, vlakt de curve af richting een ondergrens van ongeveer 8 tot 10 ms. Deze resterende tijd wordt niet langer bepaald door de logische berekeningen van de individuele entiteiten, maar wordt gedomineerd door de overhead van de kernel-aanroepen en de toegangstijden van het geheugen.

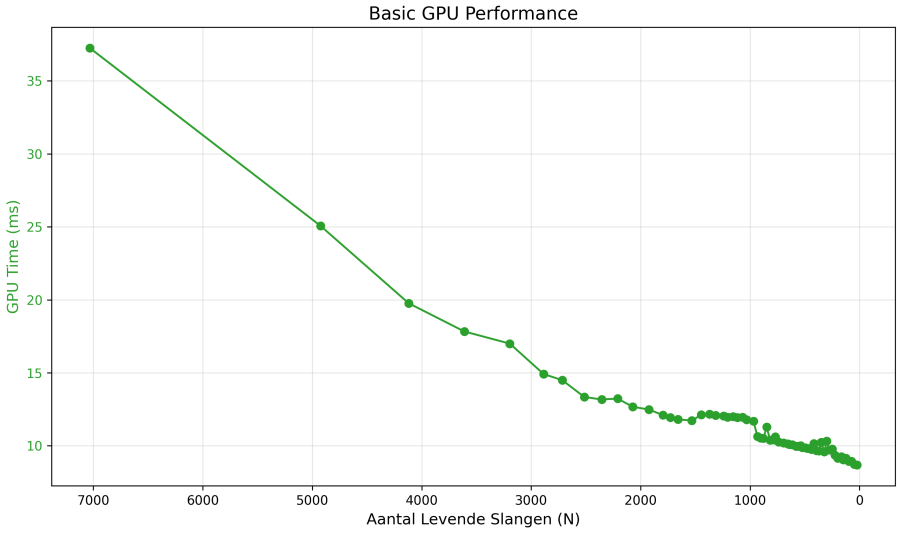


Figure 3: Snake in GPU implementatie maakt gebruik van global memory

Dit is het begin, het is duidelijk dat naarmate er minder slangen leven het programma sneller loopt. Het valt wel op dat de versnelling stagneert en zelfs even omhoog gaat tussen de 200 en 100 slangen.

3.5 optimalisatie

3.5.1 AliveList

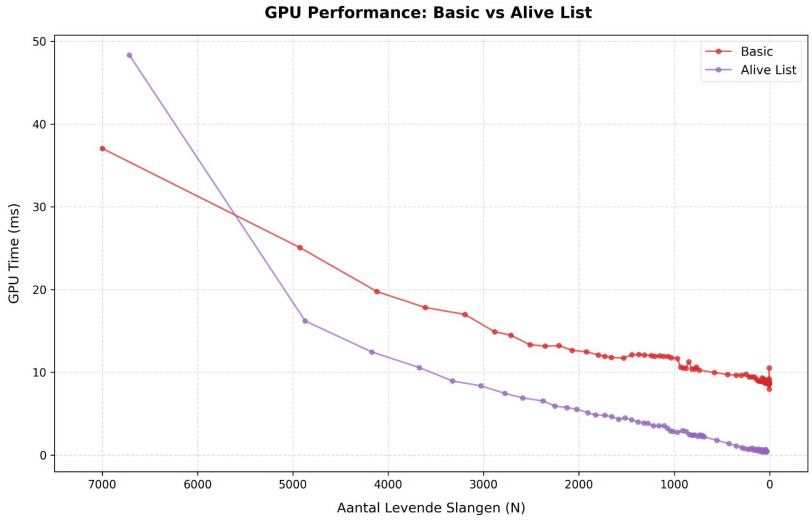


Figure 4: Vergelijking van de GPU-rekentijd met en zonder alive-list optimalisatie.

In fig. 4 valt het op dat de starttijd van alive list een pak hoger ligt dan de basic versie. Dit komt omdat er in het begin veel slangen tegelijk doodgaan. De nieuwe alive list wordt met atomic add gegenereerd, dit zorgt voor nog extra vertraging. Het punt van de alive list is om de overhead van de dode slangen te verminderen. Als alle slangen nog leven is dit gewoon een extra stap die extra tijd vergt.

uitleg alive list Bij de originele implementatie was de GPU tijd per tick soms hoger bij minder slangen. Er was één lijst voor alle slangen, of ze nog leefden of niet. Via een nieuwe lijst kan dit opgelost worden. Bij iedere tick worden de dode slangen uit de nieuwe lijst verwijderd en wordt er alleen nog verder gegaan met de levende slangen. Dit hielp enorm en het probleem van de toenemende tijd was opgelost. een beetje extra overhead om veel overhead te verwijderen was dus nodig.

```
1
2  kernel_compactAliveList<<<blocks, threads>>>(d_snakes,
       d_aliveList, aliveCount, d_newList, d_newCount);
3      // Swap new list into place
4  CUDA_CHECK(cudaMemcpy(d_aliveList, d_newList,
5      sizeof(int) * aliveCount, cudaMemcpyDeviceToDevice));
6  CUDA_CHECK(cudaMemcpy(d_aliveCount, d_newCount,
7      sizeof(int), cudaMemcpyDeviceToDevice));
8
9  __global__ void kernel_compactAliveList(
10     Snake* snakes,
11     int* aliveList, int aliveCount,
12     int* newList,   int* newCount)
13 {
14     int idx = blockIdx.x * blockDim.x + threadIdx.x;
15     if (idx >= aliveCount) return;
16     int id = aliveList[idx];
17
18     if (snakes[id].alive) {
19         int slot = atomicAdd(newCount, 1);
20         newList[slot] = id;
21     }
22 }
```

3.5.2 Fruit constant memory

Voortbouwend op vorige optimalisatie, alive list, gaan we nu kijken naar constante memory.

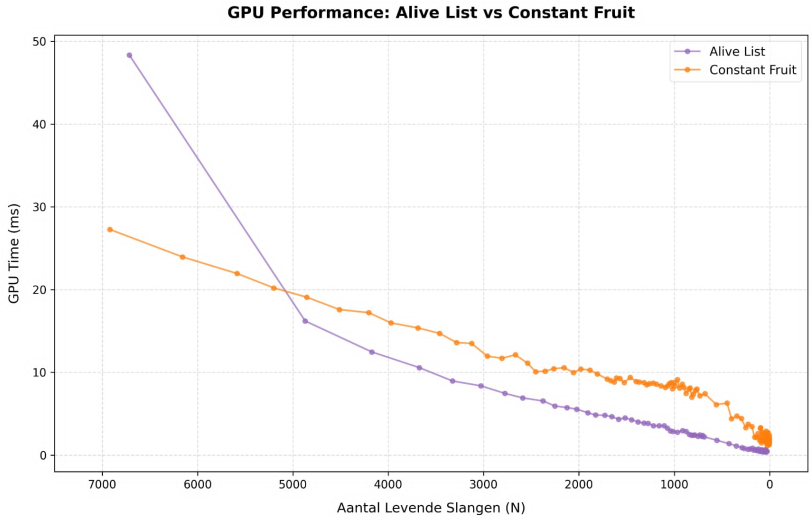


Figure 5: vergelijking alive met constant memory

De constant memory heeft een veel betere opstarttijd, zoals weergegeven in fig. 5. Dat de start-up sneller is, is omdat de moveSnakes kernel nu niet meer verantwoordelijk is om random waarden te genereren. Origineel zat de fruit logica hierin, dus iedere slang die een stuk fruit at was verantwoordelijk om een nieuw stuk fruit te maken. Dit zorgde ervoor dat de threads niet gebalanceerd waren.
Ondanks de snellere opstarttijd is constant fruit wel trager naarmate er minder slangen leven. Dit is te wijten aan het feit dat respawn eaten fruits nu iedere tick wordt opgeroepen en in de vorige versie alleen maar als er een slang een stuk fruit at (maar wel in de kernel).
Dit zou opgelost kunnen worden door een boolean te maken die true wordt als er een slang een stuk fruit heeft gegeten en pas de spawn fruit logica uitvoeren eens deze vlag hoog staat. Momenteel werken we al met een anyeaten vlag, maar deze dient alleen om niet nodeloos terug naar de GPU te copieren. Iedere tick wordt de fruit lijst gecopieerd naar de cpu en volledig doorlopen om te kijken of er van gegeten is.
Bij de nieuwe oplossing hoeven we in de respawn fruit functie alleen maar deze bool te controleren en kunnen we nodeloos veel copieren en itereren tegen gaan. Deze boolean zal wel nog altijd elke iteratie naar de CPU moeten gekopieerd worden. In vergelijking met de lijst met al het fruit is dit een stuk voordeliger. (Dit is nog niet geimplementeerd)

uitleg constant memory Fruit leeft in global memory, elke slang (lees thread) die fruitposities wou weten vroeg apart het stuk geheugen op. (Wat alle slan-

gen (dus threads) zijn die nog leven). Dit oplossen is simpel, we maken fruit constant memory. Nu zit fruit als read only op de cache van de GPU. Slangen moeten fruit eten en fruit blijft ook groeien(er is een constante voorraad fruit van 30stuks), dit kan niet met constant memory. De oplossing voor het eten is een vlag. Deze geeft aan of het fruit al dan niet gegeten is. Omdat er een constant aantal fruit is, is het simpel om de vlaggen bij te houden. Dit is gewoon een array die bestaat uit evenveel elementen als er fruit aanwezig moet zijn. De positie van het fruit bestaat uit 2 arrays, 1tje voor de X-richting en 1tje voor de Yrichting. (misschien dat per 2 steken in array betere coalescing zou geven maar maakt logica een stuk moeilijker).
Een voorbeeld: fruit 2 zit op x-coordinaat 3 en y-coordinaat 2, dit wilt zeggen dat fruitX[1] = 3 en fruitY[1] = 2 en er leeft ook een vlag in fruitEaten[1] = 0. Eens deze wordt opgegeten zal de vlag op 1 gezet worden en zal de functie spawnfruit nieuwe X en Y coordinaten genereren.
Zoals eerder al vermeld was, is dit niet de beste implementatie en hebben we wel wat snelheid verloren hierdoor.

3.5.3 SoA

De file heeft nu betere logica en constante memory, nu zal de structuur van variabelen aangepakt worden.

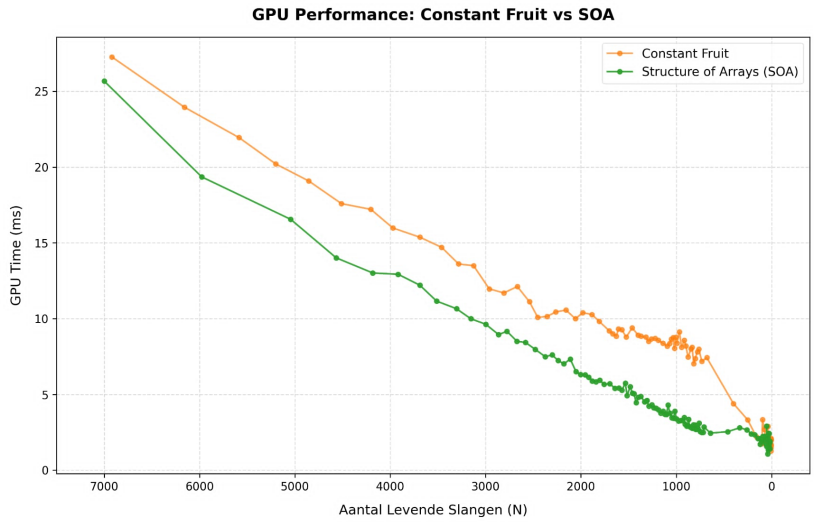


Figure 6: fruit vs SoA

In fig. 6 is de Structure of Arrays geïmplementeerd in het programma. Door een Structure of Arrays te implementeren neemt de snelheid uniform toe, voor elke hoeveelheid slangen is er een duidelijk en gelijkaardige snelheidswinst. Bij

extreem weinig slangen is er duidelijke convergentie.

uitleg SoA Als we het hier over geheugen hebben gaat het over de global memory, op de GPU dus.
Een laatste optimalisatie is het gebruikmaken van een Structure of Arrays in plaats van een Array of Structures. De naam is veelzeggend, en het is ook niet veel meer dan dat.
Als een kernel een stuk geheugen leest dan laadt niet alleen het gevraagde geheugen in maar ook geheugen die in de buurt zit. Programmas kunnen veel efficienter werken als je hier op let, maar het kan ook een bottlenek worden als alles ver uit elkaar zit.

Momenteel is iedere slang een struct en deze structs worden in een array gestoken (Array of Structs (AoS)). In de meeste gevallen zal iedere kernel die iets van 1 slang nodig heeft zal dit van alle slangen nodig hebben.
Om gelijkaardige informatie samen te houden wordt gebruik gemaakt van een SoA. Op de CPU blijven we werken met een AoS omdat dit iets overzichtelijker is.
GPU's lezen geheugen niet per byte maar per groep van bytes. Als verschillende threads tegelijk iets willen lezen kan de kernel coalescen, dit houdt in dat hij de verschillende lees-operaties samenbundelt om 1 enkele lees operatie te moeten doen. Dit kan natuurlijk alleen maar als de maximale afstand tussen 2 adressen 128 bytes bedraagt. Indien ze verder uit elkaar liggen is de kernel genoodzaakt 2 lees operaties uit te voeren.
Beschouwen we de dirX, in de originele AoS had elke snake een dirX. In deze integer zit de X richting van waar de slang naartoe gaat. Deze variabele wordt gebruikt in decideDirection. Voor AoS zal het vaak niet mogelijk zijn voor de kernel om te coalescen omdat de volledige grootte van de struct tussen 2 verschillende slangen hun dirX zit.
Maar in de SoA zitten alle dirX'en naast elkaar. Hierdoor kan de kernel makkelijker een groot blok geheugen ophalen en zal er dus ook minder blokken moeten opgehaald worden.

```
1 //voorbeeld uit de code
2 //hierin is te zien hoe we overgaan van de AoS structuur op
      de CPU naar een SoA structuur om door te sturen naar de
      GPU.
3 h_dirX[i] = snakes[i].dirX;   // alle dirX bij elkaar
4 h_dirY[i] = snakes[i].dirY;   // alle dirY bij elkaar
5
6 //eens de GPU zijn ding gedaan heeft en de data terug nodig
      is op de CPU doen we hetzelfde in omgekeerde volgorde:
7 snakes[i].dirX = h_dirX[i];
```

3.5.4 shared memory

Als laatste gaan we shared memory toevoegen aan het programma.

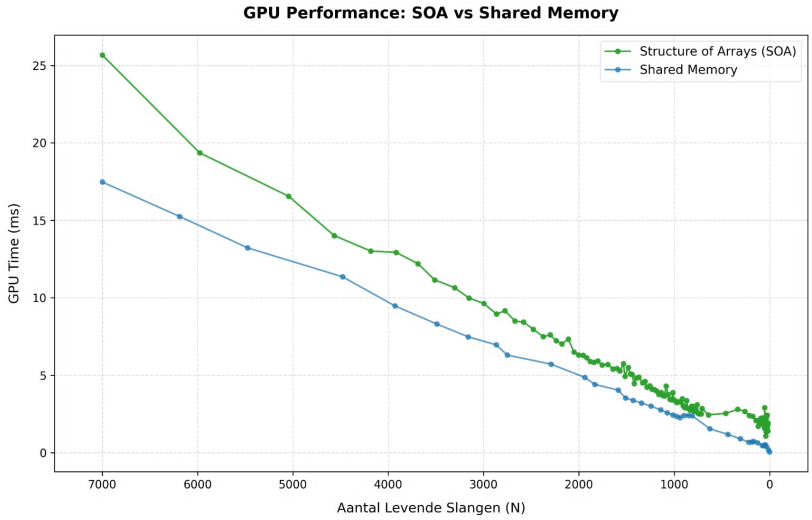


Figure 7: SoA vs shared memory

We merken opnieuw een grote winst bij de opstart waarna de shared memory consistent sneller blijft, zoals weergegeven in fig. 7. beide versies convergeren wel een beetje naarmate er minder slangen zijn maar zelfs in de buurt van nul blijft shared memory duidelijk sneller.

uitleg shared memory Het snelste gedeelde geheugen in cuda is shared memory. Deze kan gedeeld worden binnen threads in 1 blok.
Elke thread leest de volledige aliveList om de richting te bepalen en om botsingen te controleren. Dit is dus een ideale kandidaat om in shared memory te steken.
De shared list wordt per block ingeladen door alle threads, dit zorgt ervoor dat dit coalesced kan gebeuren en in paralel waardoor de lijst sneller ingeladen wordt. Er is een kleine overhead na het inladen omdat de threads gesynchroniseerd moeten worden.
Hierna blijft de logica grotendeels hetzelfde, maar in plaats van globaal met de alive list te werken wordt er per block gewerkt met de shared list.

```
1 //voorbeeld uit de code hoe de shared memory geinitialiseerd
      wordt
2 //i is de thread index, dit gaat van 0->aantal threads - 1
```

```
//blockdim = aantal threads
//thread 0 laad dus index:0, 128,..
//thread 1 laad dus index:1, 129,...
//...
//dit gebeurd per block
//zo heeft ieder block zijn eigen shared memory efficient ingeladen
//syncthreads zorgt voor een beetje overhead maar is noodzakelijk
//zonder syncthreads zou thread "a" kunnen lezen wat thread "b" nog moet schrijven

    __shared__ int s_aliveList[NUM_SNAKES];
    for (int i = threadIdx.x; i < aliveCount; i += blockDim.x)
        s_aliveList[i] = aliveList[i];
    __syncthreads();
```

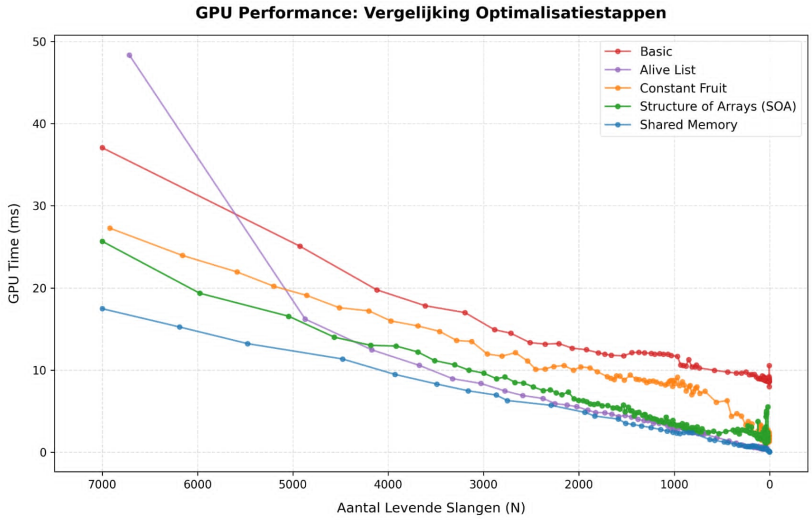


Figure 8: Volledig overzicht van alle optimalisaties

4 Code

4.1 CPU code

```
#include <SFML/Graphics.hpp>
#include <vector>
#include <string>
```

```
#include <algorithm>
#include <random>
#include <iostream>

#define NUM_SNAKES 10000
#define NUM_FRUITS 100
#define INITIAL_LEN 3
#define CELL_SIZE 5

// --- GLOBALE VARIABELEN & STRUCTS ---
struct Snake {
    std::vector<sf::Vector2i> body;
    sf::Vector2i dir;
    sf::Color color;
    bool alive;
    int score;
    int id;
};

Snake snakes[NUM_SNAKES];
sf::Vector2i fruits[NUM_FRUITS];

// Random Generator
std::random_device rd;
std::mt19937 gen(rd());

// Richting Lookup Table
sf::Vector2i directions[] = {
    {1, 0},  // Rechts
    {-1, 0}, // Links
    {0, 1},  // Onder
    {0, -1}  // Boven
};

// --- FUNCTIES ---
void spawnFruit(int id, int width, int height) {
    fruits[id].x = rand() % width;
    fruits[id].y = rand() % height;
}

void initSnake(int id, int width, int height) {

    std::uniform_int_distribution<> disX(5, width - 15);
    std::uniform_int_distribution<> disY(5, height - 5);

    snakes[id].body.clear();
    snakes[id].alive = true;
    snakes[id].score = 0;
    snakes[id].id = id;

```

```
54      sf::Vector2i head(disX(gen), disY(gen));
55      // Vul het lichaam met INITIAL_LEN segmenten op dezelfde
            plek
56      for(int i = 0; i < INITIAL_LEN; i++) {
57          snakes[id].body.push_back(head);
58      }
59
60      // Kies willekeurige startrichting uit de lookup table
61      snakes[id].dir = directions[rand() % 4];
62
63      // Willekeurige kleur
64      snakes[id].color = sf::Color(rand() % 155 + 100, rand()
            % 155 + 100, rand() % 155 + 100);
65  }
66
67  bool isSafe(sf::Vector2i pos, int width, int height) {
68      // Check muren
69      if (pos.x < 0 || pos.x >= width || pos.y < 0 || pos.y >=
            height) return false;
70
71      // Check botsing met alle andere levende slangen
72      for (int s = 0; s < NUM_SNAKES; s++) {
73          if (!snakes[s].alive) continue;
74          for (auto& segment : snakes[s].body) {
75              if (segment == pos) return false;
76          }
77      }
78      return true;
79  }
80
81  void decideDirection(int id, int width, int height) {
82      if (!snakes[id].alive) return;
83
84      sf::Vector2i head = snakes[id].body[0];
85      sf::Vector2i bestDir = snakes[id].dir;
86      float minDist = 1000000;
87
88      // Check alle 4 richtingen
89      for(auto& d : directions) {
90          // Voorkom dat de slang direct 180 graden omdraait
91          if (d.x == -snakes[id].dir.x && d.y == -snakes[id].
                dir.y) continue;
92
93          sf::Vector2i next = head + d;
94          if (isSafe(next, width, height)) {
95              // Zoek dichtstbijzijnde fruit
96              for(int f = 0; f < NUM_FRUITS; f++) {
97                  float dist = abs(next.x - fruits[f].x) + abs
                        (next.y - fruits[f].y);
98                  if (dist < minDist) {
```

```
99                      minDist = dist;
100                     bestDir = d;
101                 }
102             }
103         }
104     }
105     snakes[id].dir = bestDir;
106 }
107
108 // --- MAIN ---
109
110 int main() {
111     srand(time(NULL));
112     sf::VideoMode desktop = sf::VideoMode::getDesktopMode();
113     sf::RenderWindow window(sf::VideoMode(desktop.width *
        0.8, desktop.height * 0.8), "Snake Battle Royale CPU
        Baseline");
114
115     sf::Font font;
116     if (!font.loadFromFile("/usr/share/fonts/truetype/dejavu
        /DejaVuSans-Bold.ttf")) {
117         std::cout << "Font niet gevonden! Paden op Linux
            kunnen verschillen." << std::endl;
118     }
119
120     int SCORE_PANEL_WIDTH = 250;
121     int width = (window.getSize().x - SCORE_PANEL_WIDTH) /
        CELL_SIZE;
122     int height = window.getSize().y / CELL_SIZE;
123
124     // Initialiseer fruit en slangen
125     for (int i = 0; i < NUM_FRUITS; i++) spawnFruit(i, width
        , height);
126     for (int i = 0; i < NUM_SNAKES; i++) initSnake(i, width,
         height);
127
128     sf::Clock frameClock;
129     sf::Clock logicClock;
130
131     while (window.isOpen()) {
132         sf::Event event;
133         while (window.pollEvent(event)) {
134             if (event.type == sf::Event::Closed) window.
                close();
135         }
136
137         // --- START LOGICA & METING ---
138         logicClock.restart();
139         int aliveCount = 0;
140
```

```
141         for (int i = 0; i < NUM_SNAKES; i++) {
142             if (!snakes[i].alive) continue;
143             aliveCount++;
144 
145             decideDirection(i, width, height);
146             sf::Vector2i nextHead = snakes[i].body[0] +
                    snakes[i].dir;
147 
148             if (!isSafe(nextHead, width, height)) {
149                 snakes[i].alive = false;
150             } else {
151                 // Beweeg: voeg kop toe
152                 snakes[i].body.insert(snakes[i].body.begin()
                        , nextHead);
153 
154                 bool ate = false;
155                 for(int f = 0; f < NUM_FRUITS; f++) {
156                     if (nextHead == fruits[f]) {
157                         ate = true;
158                         spawnFruit(f, width, height);
159                         snakes[i].score++;
160                         break;
161                     }
162                 }
163                 // Als er niet gegeten is, verwijder staart
                        (slang blijft even lang)
164                 if (!ate) snakes[i].body.pop_back();
165             }
166         }
167         sf::Time logicTime = logicClock.getElapsedTime();
168         sf::Time frameTime = frameClock.restart();
169         float fps = 1.0f / frameTime.asSeconds();
170 
171         // Print metingen naar de terminal
172         static int printTimer = 0;
173         if (printTimer++ % 10 == 0) {
174             std::cout << "Alive:␣" << aliveCount
175                       << "␣|␣Logic:␣" << logicTime.
                              asMilliseconds() << "ms"
176                       << "␣|␣FPS:␣" << (int)fps << std::endl
                              ;
177         }
178 
179         // --- RENDERING ---
180         window.clear(sf::Color(20, 20, 25));
181 
182         // Teken Fruit
183         sf::CircleShape fShape((CELL_SIZE/2.0f) - 1);
184         fShape.setFillColor(sf::Color::Red);
185         for(int i = 0; i < NUM_FRUITS; i++) {
```

```
            fShape.setPosition(fruits[i].x * CELL_SIZE,
                fruits[i].y * CELL_SIZE);
            window.draw(fShape);
        }

        // Teken Slangen
        for(int s = 0; s < NUM_SNAKES; s++) {
            if (!snakes[s].alive) continue;
            sf::RectangleShape rect(sf::Vector2f(CELL_SIZE -
                1, CELL_SIZE - 1));
            rect.setFillColor(snakes[s].color);
            for(auto& seg : snakes[s].body) {
                rect.setPosition(seg.x * CELL_SIZE, seg.y *
                    CELL_SIZE);
                window.draw(rect);
            }
        }

        // Teken Sidebar Achtergrond
        sf::RectangleShape sidebar(sf::Vector2f(
            SCORE_PANEL_WIDTH, window.getSize().y));
        sidebar.setPosition(window.getSize().x -
            SCORE_PANEL_WIDTH, 0);
        sidebar.setFillColor(sf::Color(30, 30, 40, 200));
        window.draw(sidebar);

        // Leaderboard Sorteren & Tekenen (Top 20)
        std::vector<int> sortedIndices;
        for(int i = 0; i < NUM_SNAKES; i++) sortedIndices.
            push_back(i);
        std::sort(sortedIndices.begin(), sortedIndices.end()
            , [](int a, int b) {
            return snakes[a].score > snakes[b].score;
        });

        sf::Text title("LEADERBOARD", font, 22);
        title.setPosition(window.getSize().x -
            SCORE_PANEL_WIDTH + 20, 20);
        window.draw(title);

        for(int i = 0; i < 20 && i < NUM_SNAKES; i++) {
            int id = sortedIndices[i];
            sf::Text scoreText(
                "Snake " + std::to_string(id) + ": " + std::
                    to_string(snakes[id].score),
                font, 14
            );
            scoreText.setFillColor(snakes[id].alive ? snakes
                [id].color : sf::Color(100, 100, 100));
```

```
225             scoreText.setPosition(window.getSize().x -
                    SCORE_PANEL_WIDTH + 20, 60 + (i * 22));
226             window.draw(scoreText);
227         }
228
229         // HUD: Alive count
230         sf::Text aliveText("ALIVE: " + std::to_string(
                aliveCount), font, 20);
231         aliveText.setPosition(20, 20);
232         aliveText.setOutlineThickness(2);
233         aliveText.setOutlineColor(sf::Color::Black);
234         window.draw(aliveText);
235
236         window.display();
237
238         // Reset simulatie als er nog maar 1 over is
239         if (aliveCount <= 1) {
240             sf::sleep(sf::seconds(1));
241             for (int i = 0; i < NUM_SNAKES; i++) initSnake(i
                    , width, height);
242         }
243     }
244     return 0;
245 }
```

4.2 GPU code

https://github.com/andresbsmn/snek/blob/main/optimale_snek